

Airflow Avançado: Extração e Preparação de Dados com Sensors, Qualidade de Dados e Boas Práticas

Aula 18 - Extração e Preparação de Dados

Ciências de Dados e IA

IBMEC

18 de maio de 2026

- **Eventos Externos (Sensors):** Ir além do agendamento baseado em tempo (Schedule) e reagir a eventos do mundo real (disponibilidade de APIs, chegada de arquivos).
- **Data Quality (Qualidade de Dados):** Implementar travas de segurança no meio do pipeline para evitar o carregamento de dados anômalos no banco de dados.
- **Boas Práticas de Produção:**
 - Idempotência e a capacidade de reprocessamento (Backfills).
 - O perigo silencioso do *Top-Level Code* para o Scheduler do Airflow.

Eventos Externos: O que são Sensors?

Até agora, nossas DAGs eram disparadas por um cronograma fixo (`schedule='@daily'`). Mas e se o dado não estiver pronto naquele horário?

O Papel dos Sensors

Sensors são um tipo especial de Operator projetado para **esperar**. Eles pausam a execução da DAG e ficam testando uma condição continuamente até que ela seja verdadeira (ou atinja um *timeout*).

Casos de Uso Comuns:

- **HttpSensor**: Aguarda até que uma API ou site externo volte a responder com status 200 OK.
- **FileSensor**: Aguarda até que um arquivo CSV ou JSON apareça em uma pasta específica do servidor.
- **S3KeySensor**: Aguarda o upload de um arquivo num Bucket da AWS S3.

Implementando um HttpSensor

Exemplo prático: Antes de tentar extrair os dados, verificamos se a API do Yahoo Finance está operante.

```
1 from airflow.providers.http.sensors.http import HttpSensor
2
3 # Tarefa 1 (Sensor): Aguarda a API estar online
4 sensor_api = HttpSensor(
5     task_id='checar_api_yahoo',
6     http_conn_id='yahoo_api', # Conexao configurada na UI do Airflow
7     endpoint='v8/finance/chart/PETR4.SA',
8     timeout=600, # Desiste se a API nao responder em 10 minutos
9     poke_interval=60 # Faz a checagem a cada 60 segundos
10 )
11
12 # 0 Fluxo garante a espera:
13 sensor_api >> extrair_dados >> transformar >> carregar
```

O pipeline de dados não deve ser apenas um "transportador". Ele precisa garantir que não estamos injetando lixo no banco de dados corporativo (*Garbage In, Garbage Out*).

Por que validar os dados no meio do ETL?

- APIs externas mudam de formato sem aviso prévio.
- Sistemas fonte podem enviar valores nulos ou negativos por erro ou instabilidade sistêmica.
- **A Regra de Ouro:** É melhor a DAG falhar e emitir um alerta (permitindo intervenção humana) do que carregar um banco de dados de produção com valores silenciosamente errados.

Implementando uma Trava de Qualidade

A validação ocorre entre a Transformação e a Carga.

```
1 @task
2 def validar_qualidade(dados: dict):
3     """ Verifica se os dados sao consistentes antes do Load. """
4     preco = dados.get('preco_brl')
5
6     if preco is None:
7         # Lanca uma excecao para forçar a falha da Tarefa/DAG
8         raise ValueError("Data Quality Falhou: O preco retornou nulo!")
9
10    if preco <= 0:
11        raise ValueError(f"Data Quality Falhou: Preco invalido ({preco}).")
12
13    print(f"Data Quality Aprovado. Preco valido: R$ {preco}")
14    return dados # Repassa os dados para a proxima etapa (Load)
```

Boas Práticas: Idempotência e Backfills

Um pipeline de produção precisa estar preparado para falhas e reexecuções.

Idempotência

Uma operação é **idempotente** se você pode executá-la várias vezes e o resultado final no banco de dados será exatamente o mesmo que executá-la apenas uma vez.

Exemplo Prático:

- **Ruim (Não idempotente):** `INSERT INTO vendas ...` (Se rodar a DAG 2 vezes, duplica a venda).
- **Bom (Idempotente):** `UPSERT / ON CONFLICT DO UPDATE` ou deletar os dados do dia antes de inserir.

Se o seu código for idempotente, você pode fazer **Backfills** no Airflow com segurança: mandar o Scheduler reprocessar dias ou meses passados sem medo de corromper o banco.

Boas Práticas: O Perigo do Top-Level Code

Erro Comum de Iniciante: Fazer processamento, requisições HTTP ou conexões de banco de dados diretamente no corpo do arquivo Python (fora de uma `@task` ou classe de `Operator`).

```
1 from airflow import DAG
2 import requests
3
4 # TOP-LEVEL CODE: NUNCA FAÇA ISSO AQUI!
5 # Esta requisicao vai rodar no servidor inteiro a cada 30 segundos!
6 resposta = requests.get("https://api-pesada.com/dados")
7
8 with DAG('pipeline_ruim', ...):
9     # ...
```

Por que isso destrói o Airflow?

O Scheduler do Airflow faz o *parsing* (leitura) de todos os arquivos na pasta `dags/` a cada 30 segundos em média. Tudo que estiver fora de uma função de *Task* será executado em loop, sobrecarregando o servidor e derrubando sistemas parceiros com requisições fantasmas.

Resumo da Arquitetura Robusta



- **Resiliência:** Tolerante a falhas e quedas de rede (Sensors).
- **Confiabilidade:** Trava de inserção se houver anomalias (Data Quality).
- **Segurança:** Sem *Top-Level Code* e totalmente Idempotente.